

Mastering Logic and Data: if...else and Arrays in JavaScript

This presentation explores two foundational concepts in JavaScript programming: conditional logic using **if...else** statements and structured data management using Arrays. Understanding these elements is crucial for building dynamic and effective web applications.

1

Conditional execution

2

Data structures

3

Practical examples

```
if (elsray - (ffr (fiersettling);  
if (alsty: car lose >  
  
if (array fite: (hnglltractor = cantoy);  
if (lesty/stectch-C/StarLESS aftleratrack);  
  
if (elsray: (ffingn = carcctet(ratly);  
f (lstt/(lo,lop));  
f (lssty/imel arlent;/LIS);  
  
if (alsray: (flingn = arraytar > alinay);  
if (alsray fite: arbgel(featural. -adlay));  
  
if (alsray: (flingn = arraytar >  
if (alrray love:/LEO);
```

```
if (array,risten (hustal stay >  
if (array,riter: faculast cajectoy))
```

```
if (elstay cariteal.net(lonnar(rettlyr);  
if (array.(lecse iste a/lectloyr);
```

Conditional Control Flow: The if...else Statement

The `if...else` statement is JavaScript's primary mechanism for decision-making. It allows your program to execute different blocks of code depending on whether a specified condition evaluates to `true` or `false`.

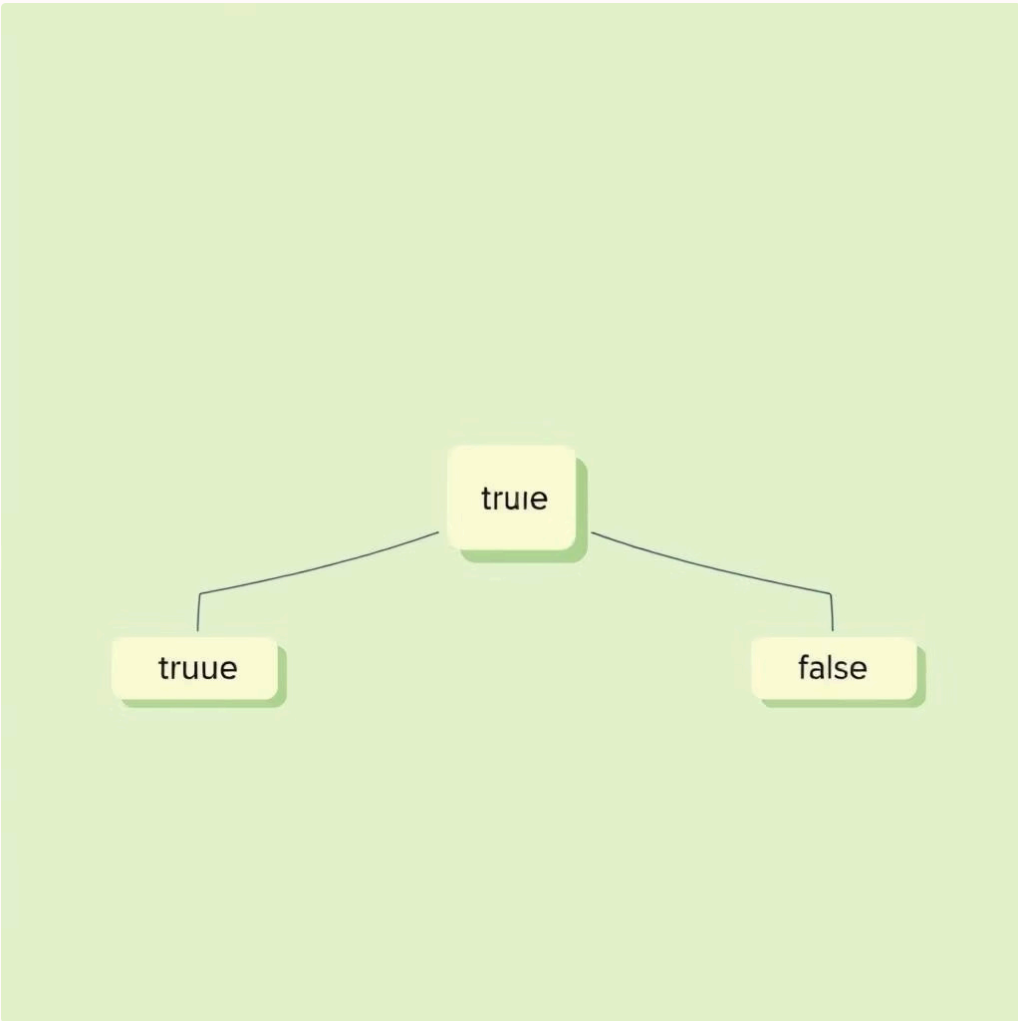
1. What is an if...else Statement?

Purpose

Executes specific blocks of code based on a condition's truthiness (boolean evaluation).

Functionality

If the initial condition is `true`, the `if` block runs. If it's `false`, the optional `else` block runs.



2. Basic Syntax and Structure

```
if (condition) {  
  // Code to execute if condition is TRUE  
} else if (anotherCondition) {  
  // Optional: Code to execute if the first condition is FALSE but this one is TRUE  
} else {  
  // Optional: Code to execute if all preceding conditions are FALSE  
}
```

Condition	if Block	else Block
A boolean expression (e.g., <code>x > 10</code>) that must be evaluated.	The required code block that runs if the condition is met (<code>true</code>).	The fallback code block that runs if the initial condition is not met (<code>false</code>).

Arrays: Organizing Your Data

While `if...else` helps with decision-making, arrays provide a powerful way to store and manage collections of related data. Think of them as organized lists where you can keep multiple items together under a single variable name.



Ordered Collections

Arrays store multiple values in a specific sequence, preserving the order in which items are added.



Zero-Indexed Access

Each item in an array is assigned a numerical index, starting from `0` for the first element.



Flexible Content

Arrays can hold various data types, including numbers, strings, objects, or even other arrays, all in one place.

Basic Syntax and Example

```
let fruits = ["Apple", "Banana", "Cherry"];

console.log(fruits[0]); // Accesses the first element: "Apple"

fruits.push("Date"); // Adds a new element to the end
console.log(fruits); // ["Apple", "Banana", "Cherry", "Date"]
```

This example demonstrates how to declare an array, access an element using its index, and add a new element using the `push()` method. Arrays are dynamic, meaning their size can change after creation.